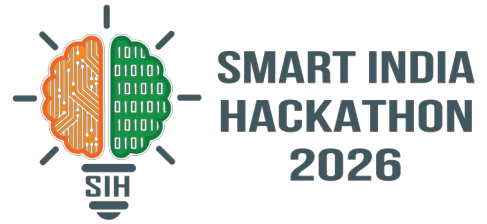


SMART INDIA HACKATHON 2026



Aasha Setu — Disaster Intelligence, Response & Situational Awareness

Problem Statement ID – **26223**

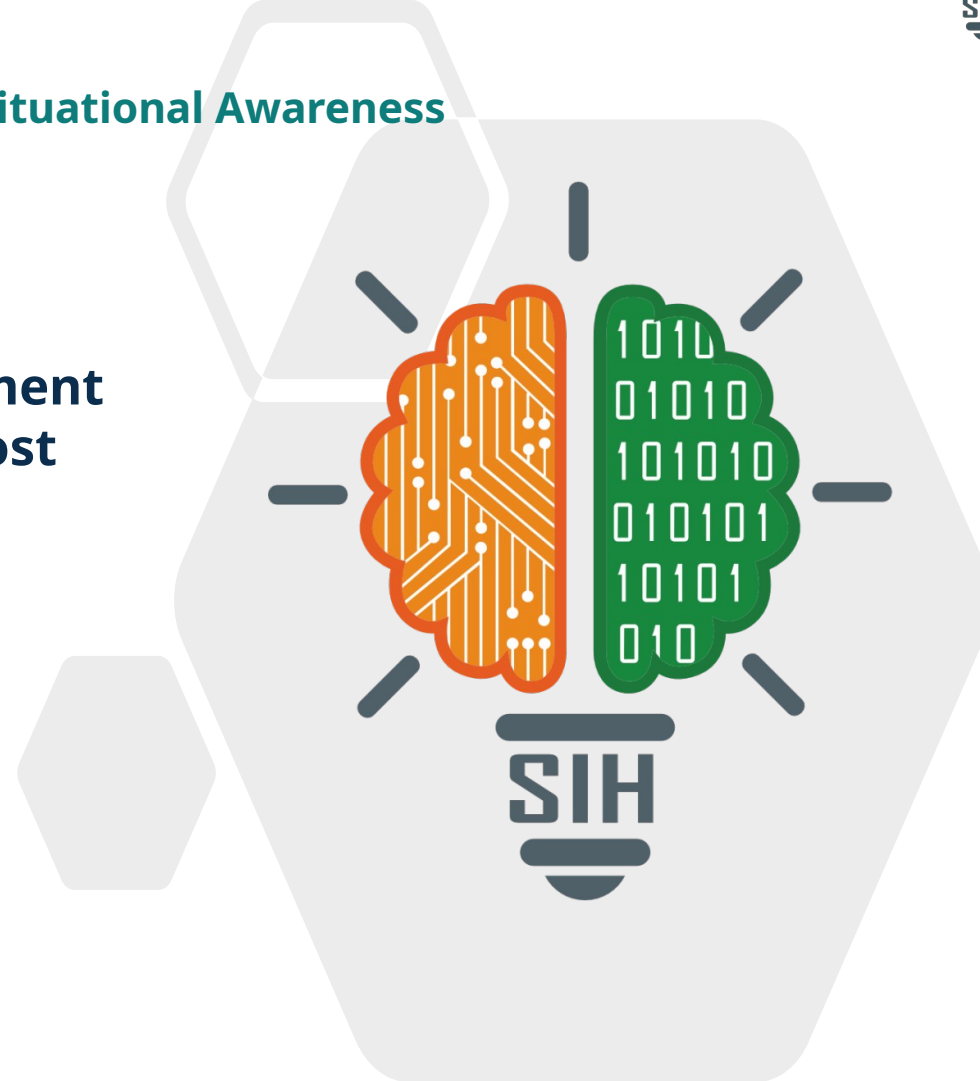
Problem Statement Title – **Disaster management and Risk mitigation System (Pre, During, Post Disaster)**

Theme – **Disaster Management**

PS Category – **Software + Hardware**

Team ID – _____

Team Name – **Innovexis**



Aasha Setu: one live loop — local risk to verified rescue

Three clients, one API, one real-time loop — a citizen's live risk becomes a verified, dispatched rescue.

THE GAP TODAY

- Hazard alerts, SOS, shelters and family status live in separate apps & hotlines
- Responders can't tell who needs help, where, or how urgently
- Unverified reports flood control rooms — no way to rank or trust them

HOW AASHA SETU FIXES IT

- One place, updated in real time — no app-switching mid-emergency
- Every report carries GPS + photo/voice + a priority score
- Human-verified before dispatch — false alarms don't burn rescue capacity

THE LIVE LOOP

1 SEE Live local risk map + rainfall / flood / cyclone / landslide forecast for your exact GPS point

2 RAISE One-tap Report, or a 3-second-hold SOS with photo, voice note and a fresh GPS fix

3 PRIORITISE Auto-scored on the Command Centre queue: severity x people affected x zone risk x report age

4 DISPATCH Operator verifies, then the nearest available responder is routed on an OSRM safe path

5 CLOSE THE LOOP Status streams back live over SSE to the citizen, their family and their "I am safe" circle

WHY IT'S DIFFERENT

► One real-time loop

SSE to every client — no Redis, no WebSocket infra to run

► Deterministic where lives are at stake

Risk & priority are auditable pure functions; AI only recommends, a human approves

► Degrades, never fails

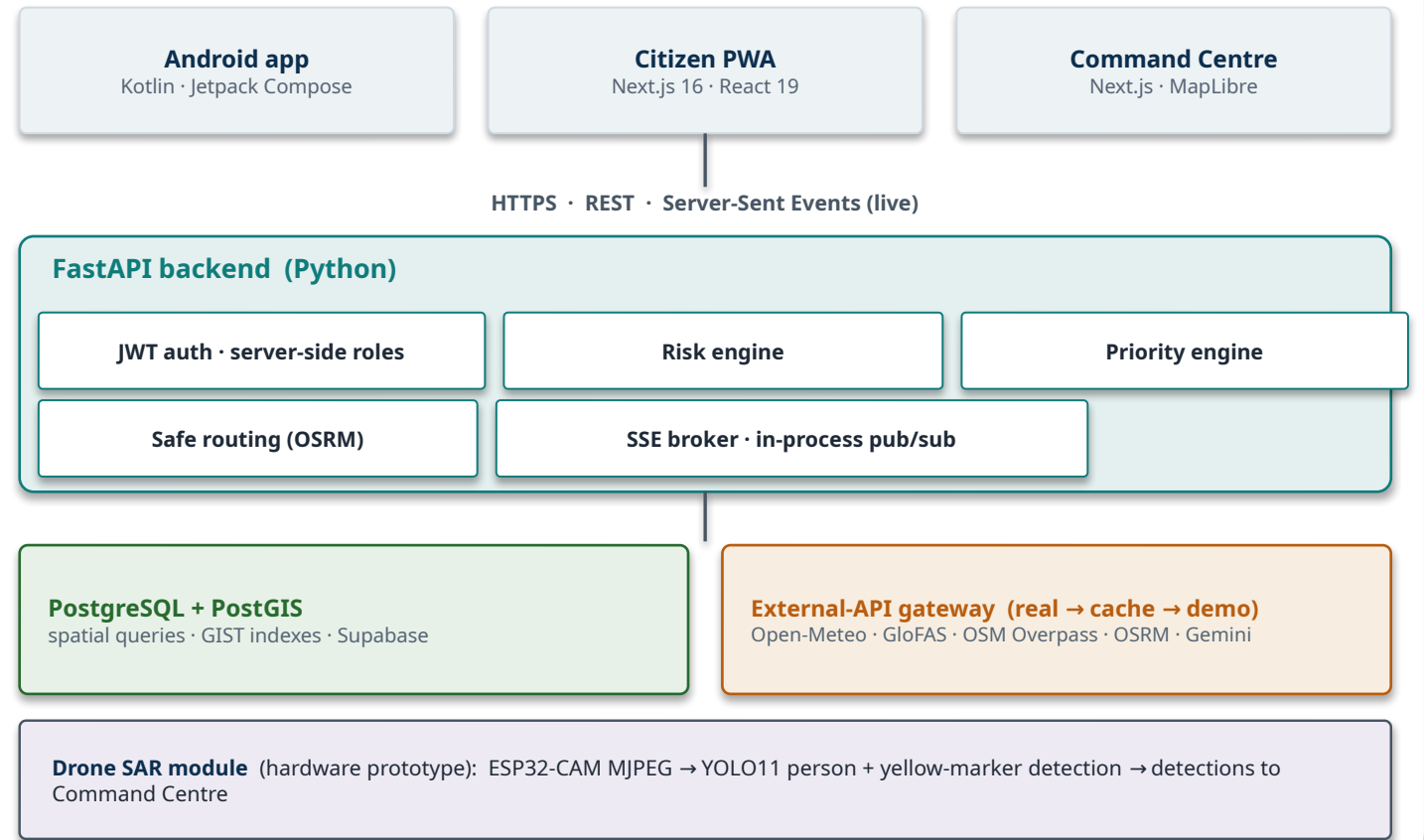
Every external API follows real → cache → demo data

► Software + hardware

Drone SAR camera (ESP32-CAM + YOLO11) feeds the same ops queue

Open-source & API-first, deterministic core, deployed on free tiers — 103 backend tests green.

SYSTEM ARCHITECTURE



Layer	Stack
Frontend	Next.js 16, React 19, TS, Tailwind v4, MapLibre GL
Android	Kotlin, Jetpack Compose, Hilt, Room, WorkManager
Backend	FastAPI, SQLAlchemy 2 + GeoAlchemy2, Pydantic v2
Database	PostgreSQL 16 + PostGIS 3.4
Realtime	Server-Sent Events + in-process broker
Hardware / CV	ESP32-CAM, Ultralytics YOLO11 (n/s)
Hosting	Vercel · Render · Supabase (free tier)

BUILD STATUS (implemented vs prototyped)

- **Running now:** citizen app ~90% of demo scope · Command Centre ~70% · live on Vercel + Render + Supabase · 103 backend tests
- **Prototyped:** drone person/marker CV, Gemini situation summary, AI confidence-scoring verification — human-in-the-loop by design
- **Method:** API-first · every integration behind the backend · deterministic core · realtime without extra infra

Zero paid-API cost, free hosting, and a graceful-degradation path for every failure mode.

TECHNICAL

- 100% open-source stack; no paid API keys
- PostGIS does the spatial maths — no hand-rolled geo code
- Boots without a .env file; 103 tests; already deployed

OPERATIONAL

- 3 roles (citizen / responder / admin) on one typed API contract
- Auto-sign-in demo path; role re-checked on every request
- Degrades gracefully if venue Wi-Fi or a provider drops

ECONOMIC

- Open-Meteo / OSM / OSRM are free at demo scale
- One codebase → 3 clients; scales on managed Postgres
- Runs on low-end Android (minSdk 26, ~99% reach)

Challenge / risk	Strategy to overcome it
Connectivity fails during a disaster	Offline reads (Room cache) + WorkManager SOS queue + downloadable offline map; drone/mesh relay as a designed fallback
An external API goes down mid-demo	real → cache → demo fallback chain, already implemented for every integration
False / duplicate reports overwhelm the control room	Confidence scoring + a mandatory human verify gate before any dispatch
Traffic spikes / horizontal scale	SSE broker contract is a drop-in for Redis; stateless JWT; GIST spatial indexes
SOS location is inaccurate	High-accuracy GPS fix captured during the 3-second hold; UI warns on coarse location
Location-sharing privacy	Opt-in only; server-authoritative roles; JWT encrypted at rest; no third-party key ever on a client

From scattered hotlines to one auditable loop — on a low-end phone, for any disaster type.

FOR CITIZENS

- Hazard forecast + risk for their exact location
- SOS with evidence that survives a dead network
- Nearest shelter / hospital; family safety status

FOR RESPONDERS & AUTHORITIES

- One prioritised queue — act on the right case first
- Verified incidents only; nearest-responder + safe route
- Area-wide alerts + live situational picture

FOR STATE DM AGENCIES

- Disaster-type-agnostic — new hazard = new type value
- Full audit trail for post-event review
- Deploys on free / low-cost managed infra

Aspect	Today	With Aasha Setu
Hazard info	Generic city-wide bulletins	Per-GPS-point, 4 hazards, live
Who needs help	Phone calls, guesswork	Geotagged reports, ranked by priority
Report trust	Unverified, duplicated	Confidence score + human verify gate
Dispatch	Manual radio coordination	Nearest responder + OSRM safe route
When network drops	Nothing gets through	Queued SOS + offline map + drone relay (designed)

SOCIAL

Faster help; families reconnected via check-ins; equal access on free OSM maps + low-end phones

ECONOMIC

No paid-API cost; open-source; reuses free hosting tiers; less duplicated field effort

SYSTEMIC

One platform for every hazard; structured data trail enables post-disaster audit & learning

One warning · one SOS · one verified rescue — that is the loop.

Built entirely on open data, open standards and open-source frameworks.

DATA & APIs (all free / open)

- Open-Meteo — weather + flood (GloFAS river discharge) · open-meteo.com
- OpenStreetMap / Overpass API · openstreetmap.org
- OSRM routing engine · project-osrm.org
- Google Gemini API · ai.google.dev

DOMAIN & STANDARDS

- NDMA National Disaster Management Guidelines · ndma.gov.in
- Common Alerting Protocol (CAP) · Sachet · sachet.ndma.gov.in
- ISRO Bhuvan geoplatform · bhuvan.nrsc.gov.in
- Ultralytics YOLO11 — aerial person detection · docs.ultralytics.com

FRAMEWORKS & PROJECT

- Frameworks: FastAPI · PostGIS · SQLAlchemy + GeoAlchemy2 · Next.js · Jetpack Compose · MapLibre GL · Ultralytics YOLO11
- Prototype code: computer-vision scripts in `ml/computer_vision/` · Gemini situation summary in `ml/nlp/`
- Project links: code repository · live citizen app · Command Centre dashboard · docs/ARCHITECTURE.md [add URLs before PDF upload]